

Survival Analysis

Efficient Bayesian Analysis of Longitudinal and Survival Data Using R-INLA
ISCB 2026, Freiburg, Germany

Denis Rustand

September 27, 2026

```
library(INLA)
library(INLAjoint)
library(ggplot2)
library(ggpubr)
library(survival)
```

Survival data

We start with simulated survival data, so the treatment effect and baseline hazard are known. Each patient has one follow-up time, an event indicator and a binary treatment variable. Censoring is generated independently of the event time.

```
set.seed(1)
n_sim <- 500
gamma_true <- 0.3 # true log-HR
mu_true <- 0.2 # true baseline rate

trt_sim <- rbinom(n_sim, 1, 0.5)
# Inverse Transform Sampling
event_times <- -log(runif(n_sim)) / (mu_true * exp(gamma_true * trt_sim))
cens_times <- runif(n_sim, 0, 4)
obs_times <- pmin(event_times, cens_times)
event <- as.integer(event_times <= cens_times)

simDat <- data.frame(id = 1:n_sim, time = obs_times,
                     event = event, trt = trt_sim)
cat("Events:", sum(event), "/", n_sim)

## Events: 172 / 500

head(simDat)
```

```
##   id      time event trt
## 1  1 2.1232352     0    0
## 2  2 1.8678323     1    0
## 3  3 1.5331336     0    1
## 4  4 1.5203942     1    1
## 5  5 0.4734263     0    0
## 6  6 0.1146259     1    1
```

The proportional hazards model

$$\lambda_i(t) = \lambda_0(t) \exp(\mathbf{X}_i^T \boldsymbol{\gamma})$$

- $\lambda_0(t)$: baseline hazard (shape of the risk over time)
- $\exp(\boldsymbol{\gamma})$: hazard ratio for a unit increase in X
- PH assumption: the hazard ratio is constant over time

In our simulated example: $\lambda_i(t) = \lambda_0(t) \exp(\boldsymbol{\gamma} \cdot \text{trt}_i)$

Unspecified baseline: the standard Cox model

The most famous approach to the proportional hazards model is the **Cox model**. Its strength lies in avoiding simultaneous estimation of the baseline hazard $\lambda_0(t)$.

Let's fit it using the standard `survival` package on our simulated data:

```
# Fit a standard Cox model using partial likelihood
M_cox <- coxph(Surv(time, event) ~ trt, data = simDat)
summary(M_cox)

## Call:
## coxph(formula = Surv(time, event) ~ trt, data = simDat)
##
##      n= 500, number of events= 172
##
##      coef exp(coef) se(coef)      z Pr(>|z|)
## trt 0.5114    1.6676   0.1534 3.334 0.000856 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##      exp(coef) exp(-coef) lower .95 upper .95
## trt      1.668      0.5997    1.235    2.252
##
## Concordance= 0.565  (se = 0.02 )
## Likelihood ratio test= 11.15  on 1 df,   p=8e-04
## Wald test               = 11.12  on 1 df,   p=9e-04
## Score (logrank) test = 11.36  on 1 df,   p=8e-04
```

The model gives us the treatment log-hazard ratio (`coef`), the hazard ratio (`exp(coef)`), and the p-values. It achieves this using a mathematical trick called **partial likelihood**: it evaluates the probability that the observed event occurred to the specific individual who failed, conditional on the risk set at that exact time. Because $\lambda_0(t)$ is common to all individuals at risk at time t , it mathematically cancels out.

Specifying the baseline in INLAjoint

`INLAjoint` uses a full likelihood and estimates the baseline hazard together with the covariate effects. This provides both hazard ratios and survival probabilities within the same Bayesian fit. A standard Cox fit can also predict survival, after estimating its cumulative baseline hazard.

Let's see how `INLAjoint` specifies the baseline, starting with simple parametric shapes and building our way up to flexible approximations.

Parametric baselines

Parametric baselines are the simplest way to make $\lambda_0(t)$ explicit. They are interpretable and fast, but the shape assumption is strong.

Exponential: constant hazard

The exponential model is the simplest: $\lambda_0(t) = \mu$. It assumes the risk of the event is constant over time. This is the baseline used to generate our first dataset. We can now check that the model recovers it.

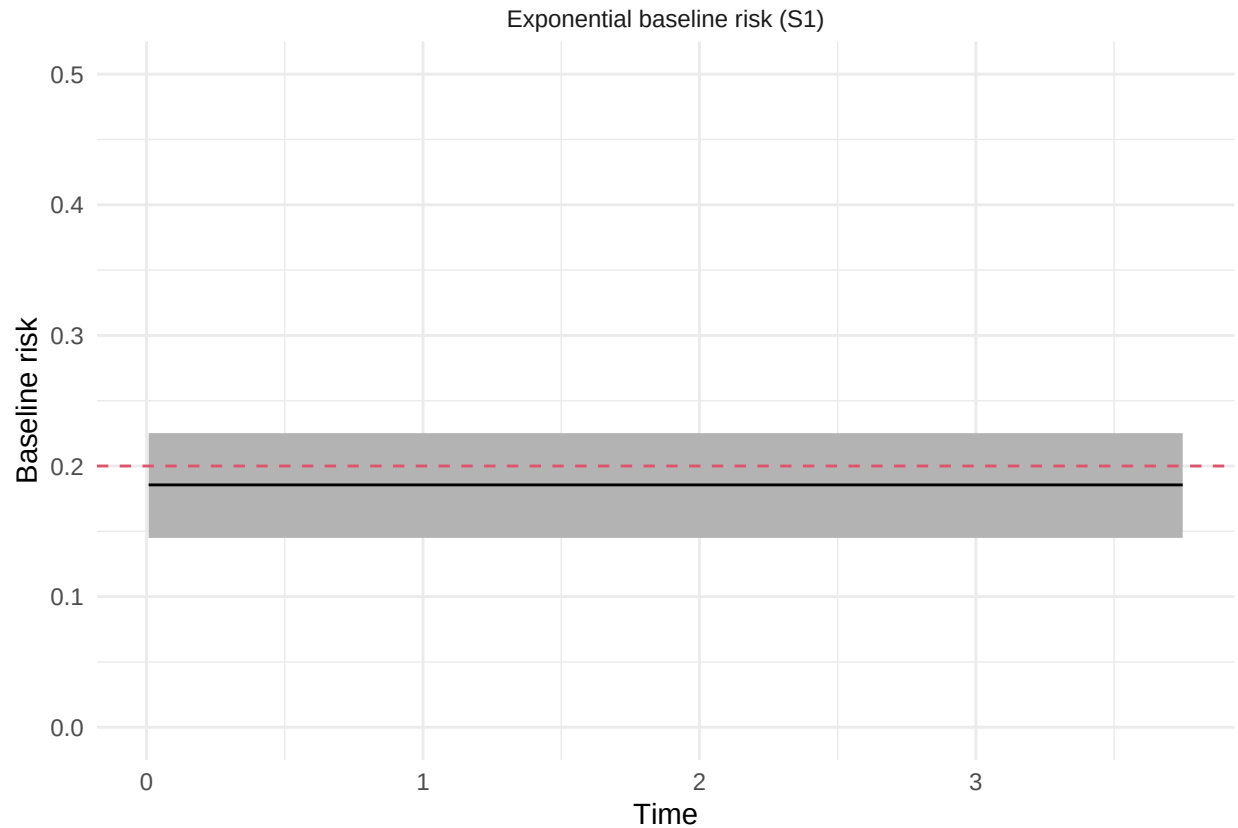
We fit this using `basRisk = "exponentialsurv"`:

```
M_exp <- joint(  
  formSurv = inla.surv(time, event) ~ trt,  
  dataSurv = simDat,  
  basRisk = "exponentialsurv"  
)  
summary(M_exp)
```

```
##  
## Survival outcome  
##               mean      sd 0.025quant 0.5quant 0.975quant  
## Exponential (rate) 0.1857 0.0206    0.1484  0.1845    0.2294  
## trt               0.5259 0.1530    0.2261  0.5259    0.8258  
##  
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)  
##                                   -421.8548                                -421.8548  
##  
## Deviance Information Criterion: 829.5587  
## Widely applicable Bayesian information criterion: 829.6559  
## Computation time: 0.77 seconds
```

```
summary(M_exp, hr=T) # hazard ratio instead of log-hazard ratio
```

```
##  
## Survival outcome  
##               exp(mean)      sd 0.025quant 0.5quant 0.975quant  
## Exponential (rate)    0.1857 0.0206    0.1484  0.1845    0.2294  
## trt                   1.7113 0.2589    1.2580  1.6917    2.2746  
##  
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)  
##                                   -421.8548                                -421.8548  
##  
## Deviance Information Criterion: 829.5587  
## Widely applicable Bayesian information criterion: 829.6559  
## Computation time: 0.77 seconds  
  
plot(M_exp)$Baseline + geom_hline(yintercept = mu_true, colour=2, linetype=2) + ylim(0,0.5)
```



Weibull: adding a shape parameter

The Weibull distribution generalizes the exponential by adding a **shape** parameter α :

$$\lambda_0(t) = \alpha \mu t^{\alpha-1}$$

Shape	Behavior	Clinical example
$\alpha < 1$	Decreasing hazard	Early mortality (post-surgery)
$\alpha = 1$	Constant (exponential)	Rare
$\alpha > 1$	Increasing hazard	Chronic disease progression

Let's simulate a scenario with an increasing hazard ($\alpha = 1.3$):

```
set.seed(1)
alpha_true <- 1.3 # increasing hazard
mu_true_w <- 0.2
gamma_true_w <- -0.5 # treatment is beneficial

trt_w <- rbinom(1000, 1, 0.5)
event_times_w <- (-log(runif(1000)) /
  (mu_true_w * exp(gamma_true_w * trt_w)))^(1 / alpha_true)
cens_times_w <- runif(1000, 0, 4)
obs_times_w <- pmin(event_times_w, cens_times_w)
event_w <- as.integer(event_times_w <= cens_times_w)
```

```
simDat_w <- data.frame(id = 1:1000, time = obs_times_w,
                      event = event_w, trt = trt_w)
```

We fit this using `basRisk = "weibullsurv"`:

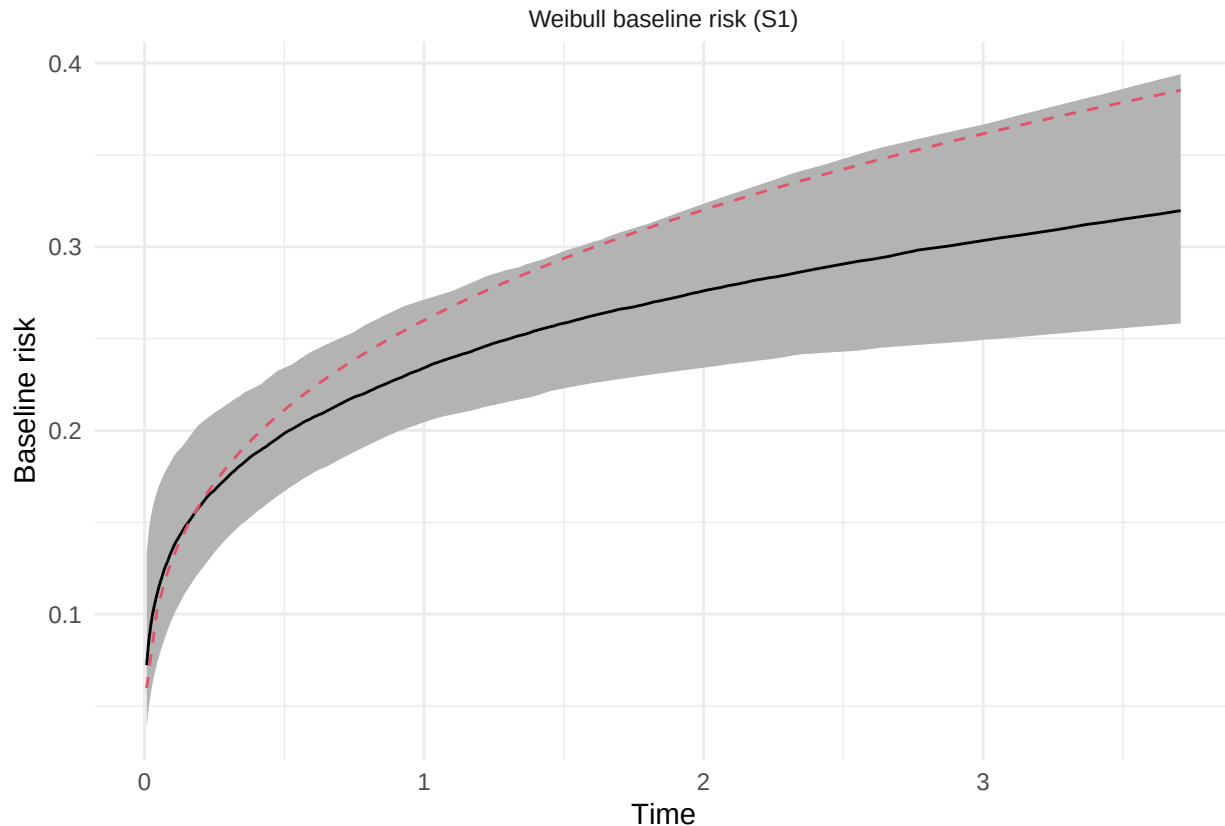
```
M_wei <- joint(
  formSurv = inla.surv(time, event) ~ trt,
  dataSurv = simDat_w,
  basRisk = "weibullsurv"
)
summary(M_wei)
```

```
##
## Survival outcome
##               mean      sd 0.025quant 0.5quant 0.975quant
## Weibull (shape) 1.2373 0.0607      1.1208  1.2365    1.3585
## Weibull (scale) 0.1897 0.0163      0.1595  0.1890    0.2235
## trt             -0.4107 0.1191     -0.6443 -0.4107   -0.1772
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##                                     -782.0927                                -782.4823
##
## Deviance Information Criterion: 1553.862
## Widely applicable Bayesian information criterion: 1553.723
## Computation time: 0.56 seconds
```

```
summary(M_wei, hr = TRUE)
```

```
##
## Survival outcome
##               exp(mean)      sd 0.025quant 0.5quant 0.975quant
## Weibull (shape) 1.2373 0.0607      1.1208  1.2365    1.3585
## Weibull (scale) 0.1897 0.0163      0.1595  0.1890    0.2235
## trt             0.6678 0.0785      0.5265  0.6631    0.8350
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##                                     -782.0927                                -782.4823
##
## Deviance Information Criterion: 1553.862
## Widely applicable Bayesian information criterion: 1553.723
## Computation time: 0.56 seconds
```

```
bw <- function(t) alpha_true*mu_true_w*t^(alpha_true-1) # true
plot(M_wei)$Baseline + geom_function(fun = bw, colour = 2, linetype=2)
```



Semi-parametric baselines: the Poisson approximation

Parametric baselines are useful, but their shapes are constrained. For a flexible baseline, `INLAjoint` uses the Poisson representation introduced in the slides (Holford, 1980; Laird & Olivier, 1981).

Each patient contributes an event count and their time at risk in each interval. With a constant hazard within an interval, the survival likelihood is proportional to a Poisson likelihood with mean **time at risk** $\times$ **hazard**. The log of time at risk is the offset.

A **random walk** prior smooths the interval-specific log-hazards:

- **RW1** (`basRisk = "rw1"`) penalizes differences between adjacent intervals.
- **RW2** (`basRisk = "rw2"`) penalizes changes in slope.

The smoothing precision is estimated with a prior. `NbasRisk` controls the number of intervals, with 15 by default. More intervals allow finer time resolution, while the random walk controls smoothness.

Under the hood, the data expansion looks like this. We use four intervals here to keep the printed table short:

```
Expanded_data <- inla.coxph(
  inla.surv(time, event) ~ trt, data = simDat,
  control.hazard = list(cutpoints = 0:4)
)

cat("Original data for patient 1:\n")
```

```
## Original data for patient 1:
```

```

print(simDat[1, c("id", "time", "event", "trt")], row.names = FALSE)

## id      time event trt
##  1 2.123235      0   0

cat("\nPoisson expanded data for patient 1:\n")

##
## Poisson expanded data for patient 1:
print(Expanded_data$data[Expanded_data$data$expand == 1,
                        c("baseline.hazard", "E..coxph", "y..coxph")],
      row.names = FALSE)

## baseline.hazard E..coxph y..coxph
##                0 1.0000000      0
##                1 1.0000000      0
##                2 0.1232352      0

```

Here, `baseline.hazard` is the start of the interval, `E..coxph` is the observed time at risk and `y..coxph` is 1 only in the interval containing the event. A censored patient has zeros throughout.

Let's fit the RW2 model on the same data as the standard Cox model. `joint()` handles the expansion internally:

```

M_rw2 <- joint(
  formSurv = inla.surv(time, event) ~ trt,
  dataSurv = simDat,
  basRisk = "rw2",
  id = "id",
  # cutpoints = 0:4,
  # NbasRisk = 15,
  # control = list(int.strategy = "eb")
)

cat("Treatment log-HR (Cox):      ", round(coef(M_cox)["trt"], 3), "\n")

## Treatment log-HR (Cox):      0.511

cat("Treatment log-HR (INLA RW2): ", round(M_rw2$summary.fixed["trt_S1", "mean"], 3))

## Treatment log-HR (INLA RW2): 0.509

# HR cox:
summary(M_cox)$conf.int

##      exp(coef) exp(-coef) lower .95 upper .95
## trt  1.667606  0.5996619  1.234605  2.252471

# HR INLA:
summary(M_rw2, hr=T)$SurvEff[[1]][2, ]

##      exp(mean)      sd 0.025quant 0.5quant 0.975quant
## trt_S1  1.68345 0.2549903  1.237109 1.664049  2.238183

```

Quite similar. Here the difference is that we do Bayesian inference and we approximate the baseline hazard.

Beyond parametric assumptions

What if the true hazard is neither constant nor strictly monotone? Can a Weibull curve follow a risk that rises and then falls?

Consider a **hump-shaped** hazard that starts near zero, peaks around $t = 2$, then declines. This happens often in health research, for example when risk is highest shortly after an operation or exposure and then drops among survivors.

Neither the exponential nor the Weibull model can capture this shape.

```
set.seed(1)
n_hump <- 1000
gamma_hump <- -0.4

# True hazard: h0(t) = scale * t * exp(-0.5*t), peaking at t = 2
# Cumulative hazard: H0(t) = scale * [4 - (2t+4)*exp(-0.5t)]
scale_h <- 1.5

trt_h <- rbinom(n_hump, 1, 0.5)

H0 <- function(t) scale_h * (4 - (2*t + 4) * exp(-0.5 * t))

gen_event <- function(g) {
  U <- runif(1)
  target <- -log(U) / exp(g)
  H0_max <- 4 * scale_h
  if (target >= H0_max) return(Inf)
  uniroot(function(t) H0(t) - target, c(1e-6, 200), tol = 1e-8)$root
}

event_times_h <- sapply(gamma_hump * trt_h, gen_event)
cens_times_h <- runif(n_hump, 0, 8)
obs_times_h <- pmin(event_times_h, cens_times_h)
event_h <- as.integer(event_times_h <= cens_times_h)

simDat_h <- data.frame(id = 1:n_hump, time = obs_times_h,
                      event = event_h, trt = trt_h)
cat("Events:", sum(event_h), "/", n_hump)
```

```
## Events: 769 / 1000
```

Let's fit all three baseline specifications:

```
MH_exp <- joint(formSurv = inla.surv(time, event) ~ trt,
               dataSurv = simDat_h, basRisk = "exponentialsurv")

MH_wei <- joint(formSurv = inla.surv(time, event) ~ trt,
               dataSurv = simDat_h, basRisk = "weibullsurv")

MH_rw2 <- joint(formSurv = inla.surv(time, event) ~ trt,
               dataSurv = simDat_h, basRisk = "rw2",
               NbasRisk = 50)
```

First, let's look at the estimated treatment effects:


```

cat("Treatment effect (true =", gamma_hump, "):\n")

## Treatment effect (true = -0.4 ):
cat("  Exponential:", round(MH_exp$summary.fixed["trt_S1", "mean"], 3),"\n")

##   Exponential: -0.369
cat("  Weibull:    ", round(MH_wei$summary.fixed["trt_S1", "mean"], 3),"\n")

##   Weibull:     -0.486
cat("  RW2:       ", round(MH_rw2$summary.fixed["trt_S1", "mean"], 3))

##   RW2:         -0.439

```

Misspecifying the baseline can also affect the treatment estimate. The baseline plot shows directly which model can recover the simulated shape.

```

# Extract baseline hazards using the plot() function data
P_exp <- plot(MH_exp)$Baseline$data
P_wei <- plot(MH_wei)$Baseline$data
P_rw2 <- plot(MH_rw2)$Baseline$data

# RW2 estimates are interval-level hazards, so plot them at interval midpoints.
rw2_width <- c(diff(P_rw2$ID), tail(diff(P_rw2$ID), 1))
rw2_time <- P_rw2$ID + rw2_width / 2

# True mathematical hazard
tt <- seq(0, 8, length = 200)
h_true <- scale_h * tt * exp(-0.5 * tt)

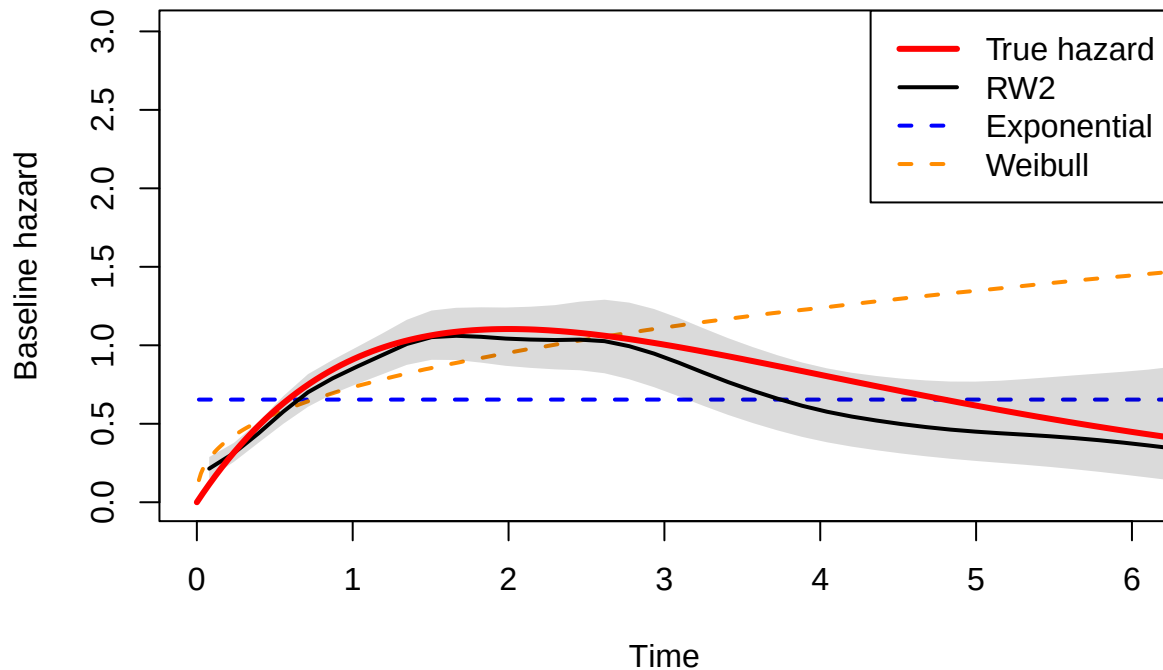
plot(P_exp$x, P_exp$y, type = "l", lwd = 2, col = "blue", lty = 2,
     xlab = "Time", ylab = "Baseline hazard",
     main = "RW2 captures a non-monotone baseline hazard",
     xlim = c(0,6),
     ylim = c(0, max(c(P_exp$upper, P_wei$upper, exp(P_rw2$q975quant), h_true)) * 1.1))

lines(P_wei$x, P_wei$y, lwd = 2, col = "darkorange", lty = 2)
polygon(c(rw2_time, rev(rw2_time)),
       c(exp(P_rw2$q025quant), rev(exp(P_rw2$q975quant))),
       col = rgb(0, 0, 0, 0.15), border = NA)
lines(rw2_time, exp(P_rw2$q5quant), lwd = 2, col = "black")
lines(tt, h_true, col = "red", lwd = 3)

legend("topright",
     c("True hazard", "RW2", "Exponential", "Weibull"),
     col = c("red", "black", "blue", "darkorange"),
     lwd = c(3, 2, 2, 2), lty = c(1, 1, 2, 2))

```

RW2 captures a non-monotone baseline hazard



The visual comparison is clear:

- The **exponential** model draws a flat line, assuming constant risk.
- The **Weibull** model is more flexible, but it is still forced to draw a monotone curve.
- The **RW2** model closely follows the true peaked risk dynamic.

We keep the hump-shaped data and the fitted RW2 model for predictions and prior sensitivity below.

Survival predictions

We use the fitted baseline and treatment effect to compute absolute survival probabilities.

Let's predict the survival trajectory for two hypothetical patients, one for each treatment arm:

```
NewS <- data.frame(id = c(1, 2),
                   time = c(0, 0),
                   event = c(0, 0),
                   trt = c(0, 1))

# Predict out to 5 years
P_hump <- predict(MH_rw2, newData = NewS, horizon = 5,
                 survival = TRUE, id = "id")

P_hump$PredS$Group <- factor(ifelse(P_hump$PredS$id == 1, "trt = 0", "trt = 1"))

ggplot(P_hump$PredS, aes(x = time, y = Surv_quant0.5, color = Group, fill = Group)) +
  geom_line(linewidth = 1) +
  geom_ribbon(aes(ymin = Surv_quant0.025, ymax = Surv_quant0.975),
            alpha = 0.15, color = NA) +
```

```
labs(x = "Time (years)", y = "Survival probability",
     title = "Predicted survival (RW2 model)") + ylim(0, 1)
```



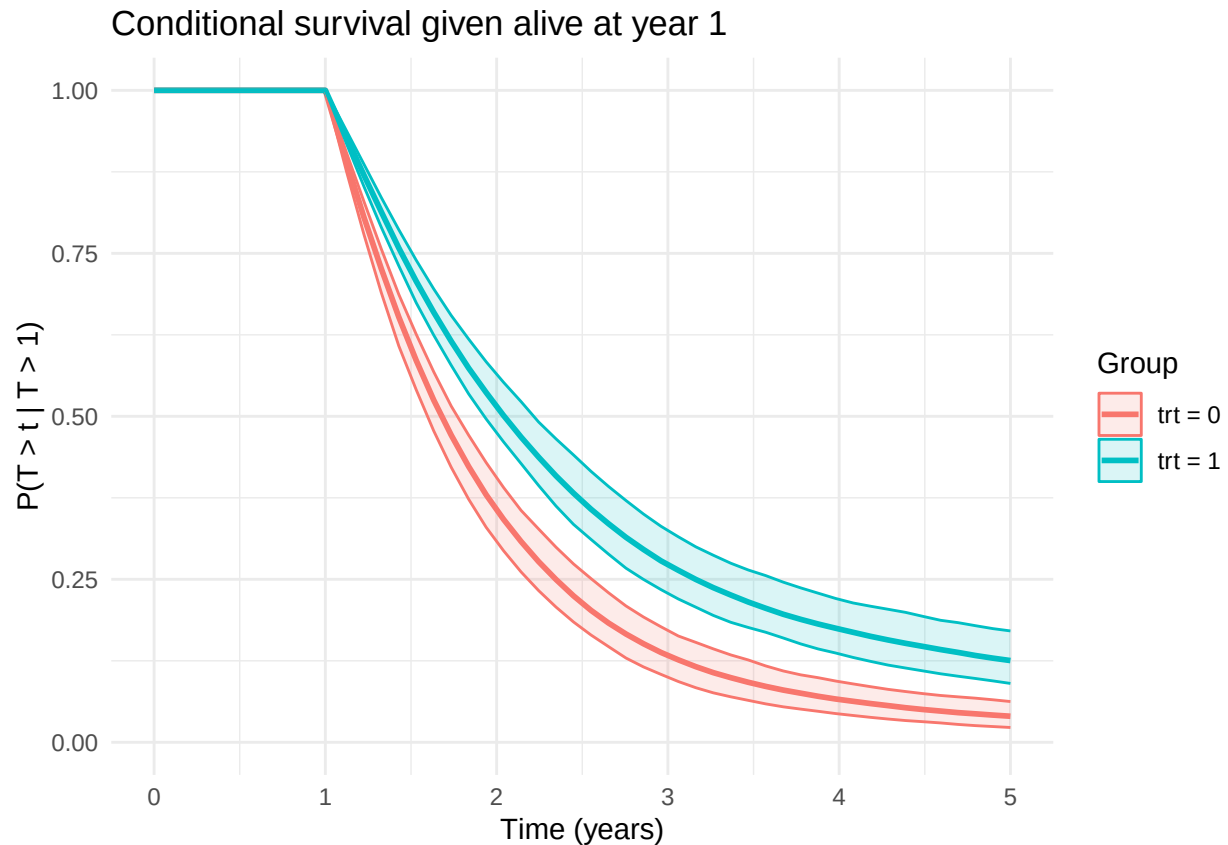
Conditional survival

Clinicians often ask dynamic questions: *If a patient has already survived past year 1, what is their updated prognosis?*

We can compute conditional survival by setting `Csurv = 1`:

```
set.seed(1)
P_cond <- predict(MH_rw2, newData = NewS, horizon = 5,
                  survival = TRUE, id = "id", Csurv = 1)
P_cond$PredS$Group <- factor(ifelse(P_cond$PredS$id == 1, "trt = 0", "trt = 1"))

ggplot(P_cond$PredS, aes(x = time, y = Surv_quant0.5, color = Group, fill = Group)) +
  geom_line(linewidth = 1) +
  geom_ribbon(aes(ymin = Surv_quant0.025, ymax = Surv_quant0.975),
            alpha = 0.15) +
  labs(x = "Time (years)", y = "P(T > t | T > 1)",
       title = "Conditional survival given alive at year 1") +
  ylim(0, 1) + theme_minimal()
```



The survival probability is 100% up to year 1 because we are conditioning on the fact that the patient is confirmed alive at that landmark.

Priors for the parameters

How much do the prior choices affect the treatment effect and baseline?

Fixed effects

The default Gaussian prior has mean 0 and variance 100 ($\text{prec} = 0.01$), on the log-hazard ratio scale.

```
MH_rw2$priors_used$priorFixed
```

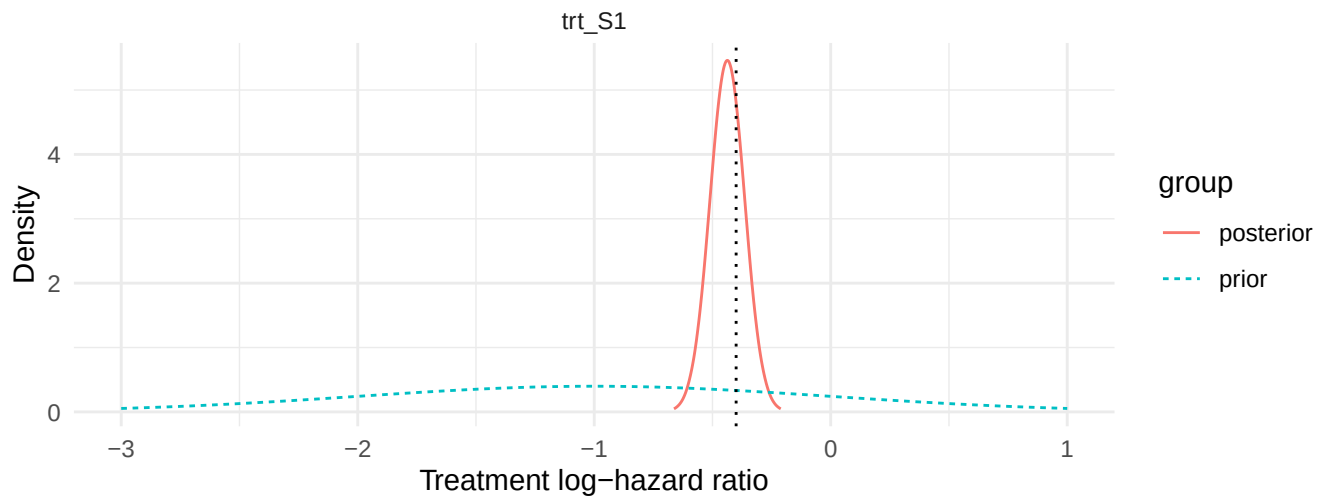
```
## $mean
## [1] 0
##
## $prec
## [1] 0.01
```

First try a tighter prior, $N(-1, 1)$. The true treatment log-hazard ratio, -0.4, is well within its range. `priors = TRUE` overlays the prior and posterior densities:

```
MH_rw2_inf <- joint(
  formSurv = inla.surv(time, event) ~ trt,
  dataSurv = simDat_h, basRisk = "rw2",
  control = list(priorFixed = list(mean = -1, prec = 1))
)
```

```
P_prior_inf <- plot(MH_rw2_inf, priors = TRUE)$Outcomes[[1]]
P_prior_inf$data <- subset(P_prior_inf$data, Effect == "trt_S1")
P_prior_inf +
  geom_vline(xintercept = gamma_hump, linetype = 3) +
  labs(x = "Treatment log-hazard ratio", title = "A moderately informative prior")
```

A moderately informative prior



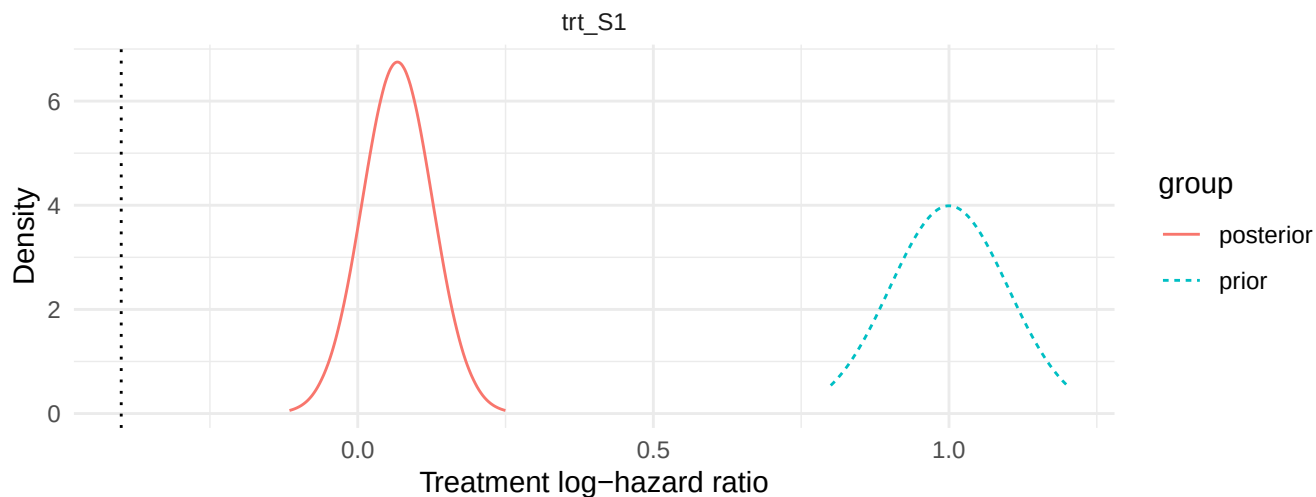
What happens when the prior is confident but points the wrong way?

Now use $N(1, 0.01)$, where 0.01 is the variance. This prior strongly favors a harmful treatment effect, although the true effect is protective.

```
MH_rw2_bad <- joint(
  formSurv = inla.surv(time, event) ~ trt,
  dataSurv = simDat_h, basRisk = "rw2",
  control = list(priorFixed = list(mean = 1, prec = 100))
)

P_prior_bad <- plot(MH_rw2_bad, priors = TRUE)$Outcomes[[1]]
P_prior_bad$data <- subset(P_prior_bad$data, Effect == "trt_S1")
P_prior_bad +
  geom_vline(xintercept = gamma_hump, linetype = 3) +
  labs(x = "Treatment log-hazard ratio", title = "A strong conflicting prior")
```

A strong conflicting prior



The vertical dotted line marks the truth. A strong conflicting prior can pull the posterior away from it. In general, when data information is sufficient and prior is not conflicting, it has a negligible impact on the posterior.

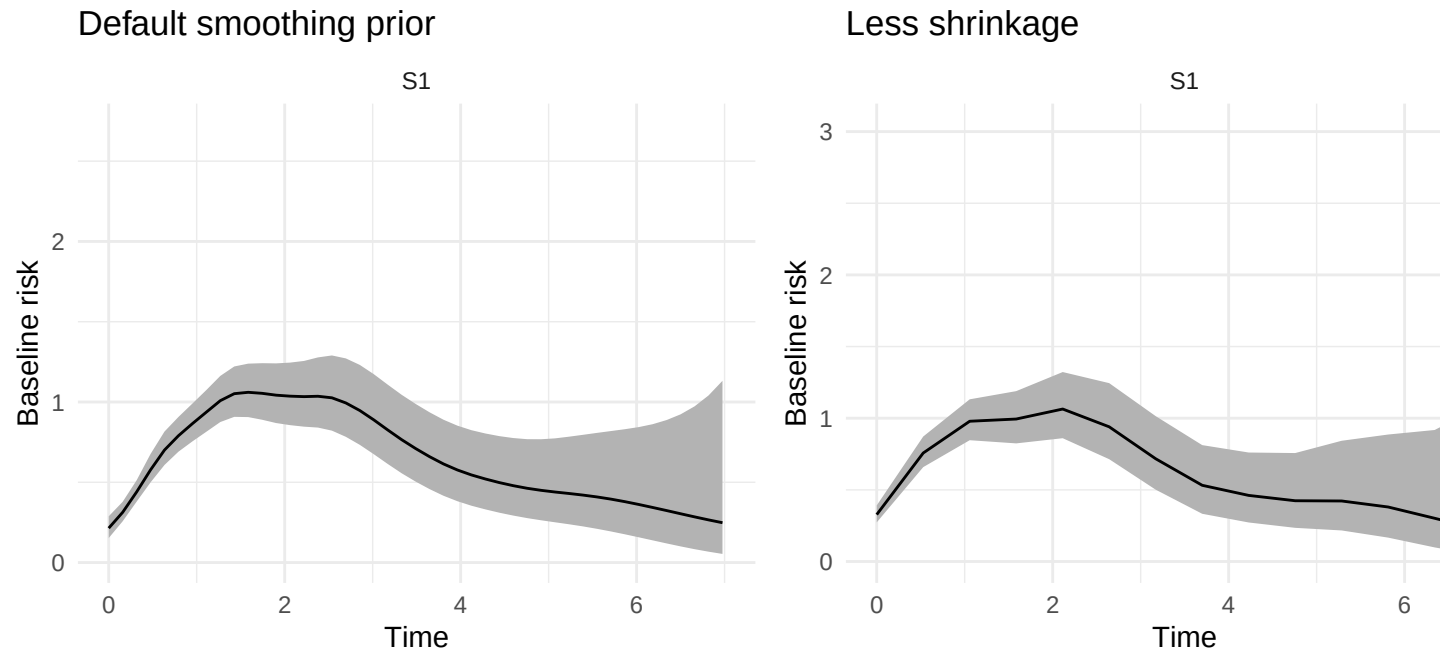
Baseline smoothness

The random walk precision has a PC prior. `priorRW = c(sigma0, p)` specifies $P(\sigma > \sigma_0) = p$, where σ is the scale of the scaled random walk. The default is `c(0.5, 0.01)`. Allowing a larger scale weakens the shrinkage toward the random walk's smooth limiting shape.

```
MH_rw2$priors_used$priorRW
```

```
## [1] 0.50 0.01
```

```
MH_rw2_prior <- joint(  
  formSurv = inla.surv(time, event) ~ trt,  
  dataSurv = simDat_h, basRisk = "rw2",  
  control = list(priorRW = c(1, 0.5))  
)  
  
ggarrange(  
  plot(MH_rw2)$Baseline + ggtitle("Default smoothing prior") + xlim(0,7),  
  plot(MH_rw2_prior)$Baseline + ggtitle("Less shrinkage") + xlim(0,7),  
  ncol = 2  
)
```



Time-dependent covariates and follow-up reconstruction

The interval format is also useful when covariates change during follow-up.

One line versus several lines

We reuse the Weibull simulation and model fitted earlier in the usual one-line-per-patient format.

```
M_weib_one <- M_wei
```

Now split each patient's follow-up at a common grid of half-year cutpoints. The function `inla.coxph()` creates the same type of interval table that we used for the Poisson approximation.

```
Itv_weib <- inla.coxph(
  inla.surv(time, event) ~ trt,
  data = simDat_w,
  control.hazard = list(cutpoints = seq(0, 4, by = 0.5))
)

td_split <- Itv_weib$data
td_split$start <- td_split$baseline.hazard
td_split$stop <- td_split$baseline.hazard + td_split$E..coxph
```

Look at the first patient in both formats.

```
cat("Original data for patient 1:\n")
```

```
## Original data for patient 1:
```

```
print(simDat_w[1, c("id", "time", "event", "trt")], row.names = FALSE)
```

```
## id      time event trt
## 1 2.427109      1    0
```

```
cat("\nExpanded data for patient 1:\n")
```

```
##
```

```
## Expanded data for patient 1:
```

```
print(head(td_split[td_split$id == simDat_w$id[1],
                c("id", "start", "stop", "y..coxph", "trt")], 10),
      row.names = FALSE)
```

```
## id start      stop y..coxph trt
##  1  0.0 0.500000      0  0
##  1  0.5 1.000000      0  0
##  1  1.0 1.500000      0  0
##  1  1.5 2.000000      0  0
##  1  2.0 2.427109      1  0
```

Each line is a small piece of follow-up. The patient is at risk from `start` to `stop`. The event indicator is 0 for all intervals before the event or censoring time, and 1 only in the final interval if the event is observed.

We can now refit the same Weibull model using left truncation and right censoring on the expanded data.

```
M_weib_split <- joint(
  formSurv = inla.surv(stop, y..coxph, truncation = start) ~ trt,
  dataSurv = td_split,
  basRisk = "weibullsurv"
)

round(rbind(
  one_line = M_weib_one$summary.fixed["trt_S1", c("mean", "0.025quant", "0.975quant")],
  split_followup = M_weib_split$summary.fixed["trt_S1", c("mean", "0.025quant", "0.975quant")]
), 4)
```

```
##              mean 0.025quant 0.975quant
## one_line        -0.4107    -0.6443    -0.1772
## split_followup -0.4107    -0.6443    -0.1772
```

The two estimates match up to numerical precision. We changed the data format, not the survival information.

Adding a time-dependent covariate

Covariates can now vary by interval. Here we allow the treatment effect to differ after year 1. The simulated effect is constant, so the additional coefficient should include zero.

```
td_split$after_year1 <- as.integer(td_split$start >= 1)
td_split$trt_after_year1 <- td_split$trt * td_split$after_year1
M_weib_td <- joint(
  formSurv = inla.surv(stop, y..coxph, truncation = start) ~
    trt + trt_after_year1,
  dataSurv = td_split,
  basRisk = "weibullsurv"
)

summary(M_weib_td)
```

```
##
```

```
## Survival outcome
```

```
##              mean      sd 0.025quant 0.5quant 0.975quant
## Weibull (shape) 1.2420 0.0698      1.1083  1.2410      1.3818
```



```
## Weibull (scale)  0.1890 0.0169      0.1576  0.1883      0.2241
## trt             -0.3974 0.1728     -0.7359 -0.3976     -0.0581
## trt_after_year1 -0.0280 0.2164     -0.4523 -0.0280      0.3964
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##                                         -785.9176                      -786.3065
##
## Deviance Information Criterion:  1555.796
## Widely applicable Bayesian information criterion:  1555.592
## Computation time: 0.89 seconds
```

Read this model as follows:

- `trt` is the treatment effect before year 1
- `trt_after_year2` is the extra treatment effect after year 1
- the sum of both coefficients is the treatment effect after year 1

In real data, we would often choose cutpoints at the times where the covariate is measured or updated. Between two measurement times, the covariate is treated as constant. This is simple, transparent and directly connected to the likelihood.

Competing risks

So far there was one event type. With competing risks, the first event can have different causes, each with its own hazard. We reuse the hump-shaped event times as cause 1 and add an independent cause 2 with a constant hazard and no treatment effect. The first event ends follow-up.

```
set.seed(2)
n_cr <- nrow(simDat_h)
other_rate <- 0.06
other_times <- -log(runif(n_cr)) / other_rate

event_time_cr <- pmin(event_times_h, other_times, cens_times_h)
cause1 <- as.integer(event_times_h <= other_times & event_times_h <= cens_times_h)
cause2 <- as.integer(other_times < event_times_h & other_times <= cens_times_h)

crDat <- data.frame(
  id = simDat_h$id,
  eventTime = event_time_cr,
  cause1 = cause1,
  cause2 = cause2,
  trt = simDat_h$trt
)

c(cause1 = sum(crDat$cause1), cause2 = sum(crDat$cause2),
  censored = sum(crDat$cause1 == 0 & crDat$cause2 == 0))

##   cause1   cause2 censored
##     708     82     210
```

A competing risks model is a list of cause-specific formulas. Both share the observed follow-up time and use a different event indicator:

```
M_cr <- joint(
  formSurv = list(
    inla.surv(eventTime, cause1) ~ trt,
    inla.surv(eventTime, cause2) ~ trt
  )
)
```

```

),
dataSurv = crDat,
basRisk = "rw2",
id = "id"
)

summary(M_cr)

##
## Survival outcome (S1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1  0.3737 0.2002      0.1201   0.3292    0.8916
## trt_S1                    -0.4287 0.0767      -0.5791  -0.4287   -0.2782
##
## Survival outcome (S2)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S2  0.0224 0.0555      0.0001   0.0055    0.1575
## trt_S2                    -0.2117 0.2223      -0.6476  -0.2117    0.2242
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##                               -2680.715                               -2680.319
##
## Deviance Information Criterion:  5264.352
## Widely applicable Bayesian information criterion:  5264.314
## Computation time: 1.74 seconds

```

The treatment hazard ratios describe each cause among patients still free of any event. Their true values are -0.4 for cause 1 and 0 for cause 2.

Cumulative incidence

Treating the competing event as censored is appropriate when fitting a **cause-specific hazard**. For the **probability** of a particular cause, we need all causes: a competing event prevents the event of interest from occurring later.

The cumulative incidence function (CIF) is

$$F_k(t) = P(T \leq t, D = k) = \int_0^t S(u) \lambda_k(u) du,$$

where $S(u)$ is survival free of **all** events. Thus $S(t) + F_1(t) + F_2(t) = 1$. Using 1 - KM with competing events censored does not estimate the CIF.

```

NewCR <- data.frame(id = c(1, 2), eventTime = c(0, 0),
                    cause1 = c(0, 0), cause2 = c(0, 0), trt = c(0, 1))

set.seed(12)
P_cr <- predict(M_cr, newData = NewCR, horizon = 4, CIF = TRUE, id = "id")

```

CIF = TRUE to switch from cause-specific hazard (default) to Cumulative Incidence Functions.

```

df_cr <- P_cr$PredS
df_cr$Treatment <- factor(ifelse(df_cr$id == 1, "trt = 0", "trt = 1"))
df_cr$Event <- ifelse(grepl("S_1", df_cr$Outcome), "Cause 1", "Cause 2")

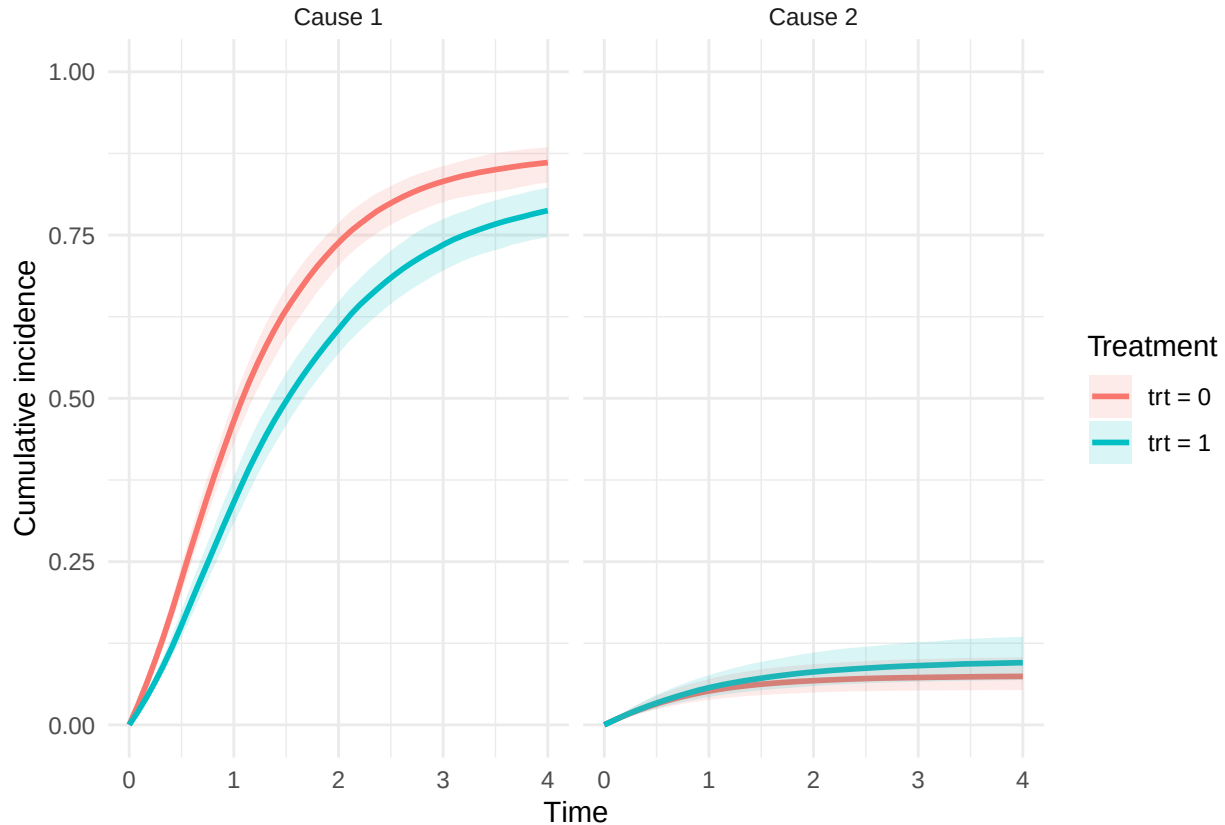
ggplot(df_cr, aes(x = eventTime, y = CIF_quant0.5,

```

```

    color = Treatment, fill = Treatment)) +
  facet_wrap(~Event) +
  geom_line(linewidth = 1) +
  geom_ribbon(aes(ymin = CIF_quant0.025, ymax = CIF_quant0.975),
    alpha = 0.15, color = NA) +
  labs(x = "Time", y = "Cumulative incidence") +
  ylim(0, 1) + theme_minimal()

```



The CIF answers the probability question for each cause. Its value depends on both cause-specific hazards, not just the hazard for that cause. The bands are pointwise 95% credible intervals.

Frailty models

The models so far assume patients are independent once we know their covariates. Suppose the patients are grouped, for example enrolled across many centers.

Centers differ in ways we do not measure: care quality, equipment, local patient mix. Two patients on the same arm but in different centers do not share the same background risk.

A frailty term adds a random effect to the hazard, shared by all patients in a center. It is a random intercept on the log-hazard, and it absorbs the between-center spread that the covariates leave behind.

$$\lambda_i(t) = \lambda_0(t) \exp(\gamma \cdot \text{trt}_i + v_{g(i)}), \quad v_g \sim N(0, \sigma_v^2).$$

We build an illustrative multi-center dataset: 500 patients over 50 centers, each center carrying its own hidden risk shift.

```

set.seed(5)
n_f <- 500
n_centers <- 50
center <- rep(1:n_centers, each = n_f / n_centers)

# One latent risk shift per center, sigma_v = 0.5 is its spread.
sigma_v <- 0.5
center_effect <- rnorm(n_centers, 0, sigma_v)
v_i <- center_effect[center]
trt_f <- rbinom(n_f, 1, 0.5)

# Hazard carries treatment (-0.50, so trt = 1 lowers risk) plus the center shift.
event_times_f <- -log(runif(n_f)) / (0.20 * exp(-0.50 * trt_f + v_i))
censor_times_f <- runif(n_f, 0, 5)

frailtyDat <- data.frame(
  id = 1:n_f,
  time = pmin(event_times_f, censor_times_f),
  event = as.integer(event_times_f <= censor_times_f),
  trt = trt_f,
  center = center
)

```

The frailty enters the formula as (1 | center). The grouping variable also becomes the id.

```

M_frailty <- joint(
  formSurv = inla.surv(time, event) ~ trt + (1 | center), # random center intercept on the hazard
  dataSurv = frailtyDat,
  id = "center",
  basRisk = "rw2"
)

summary(M_frailty, sdcor = TRUE, hr = TRUE) # sdcor prints the frailty as a standard deviation

```

```

##
## Survival outcome
##
## exp(mean)      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)  0.1010 0.0930      0.0096   0.0739   0.3551
## trt                    0.5918 0.0976      0.4229   0.5837   0.8060
##
## Frailty term standard deviation
##
## mean      sd 0.025quant 0.5quant 0.975quant
## IDIntercept 0.4995 0.0864      0.3507   0.4925   0.6896
##
## log marginal-likelihood (integration)      log marginal-likelihood (Gaussian)
##
## -837.6276                                -837.2670
##
## Deviance Information Criterion: 1627.884
## Widely applicable Bayesian information criterion: 1628.148
## Computation time: 0.95 seconds

```

Reading the output:

- the treatment hazard ratio is conditional on center frailty, a within-center comparison. Treatment was simulated at -0.50 on the log-hazard, so a ratio near 0.6.

- the frailty standard deviation measures residual center spread, simulated at 0.5.
- a positive center frailty means higher event risk for everyone in that center.

```
head(M_frailty$summary.random$IDIntercept_S1)
```

##	ID	mean	sd	0.025quant	0.5quant	0.975quant	mode	kld
## 1	1	-0.06277681	0.3682521	-0.7845377	-0.06277681	0.6589841	-0.06277681	0
## 2	2	0.54152701	0.3840235	-0.2111451	0.54152701	1.2941992	0.54152701	0
## 3	3	-0.38038352	0.3895757	-1.1439378	-0.38038352	0.3831707	-0.38038352	0
## 4	4	0.10351337	0.3642874	-0.6104768	0.10351337	0.8175035	0.10351337	0
## 5	5	0.36848274	0.3667892	-0.3504109	0.36848274	1.0873764	0.36848274	0
## 6	6	-0.12539633	0.3913309	-0.8923909	-0.12539633	0.6415982	-0.12539633	0

The same frailty structure handles recurrent events, where one patient contributes several event times. It also combines with time-dependent covariates: split each patient's follow-up into intervals, attach the current covariate value to each interval and keep the frailty in the formula.

Shared frailty joint model

A frailty can do more than absorb clustering. It can link two event processes.

Suppose patients have a recurrent event, repeated hospitalizations or repeated new lesions, and also a terminal event, death. A patient with high unobserved risk for the recurrent event often has high unobserved risk of death too.

When both processes depend on the same frailty, death informatively ends observation of recurrent events. A joint model accounts for this dependence.

The shared frailty joint model fits both processes together. One latent risk per patient enters both hazards, scaled by a parameter that measures how strongly the two are tied.

$$\begin{aligned}\lambda_{i1}(t) &= \lambda_{01}(t) \exp\{\gamma_1 \text{trt}_i + u_i\}, \\ \lambda_{i2}(t) &= \lambda_{02}(t) \exp\{\gamma_2 \text{trt}_i + \varphi u_i\}, \\ u_i &\sim N(0, \sigma_u^2).\end{aligned}$$

The parameter φ is the link. If $\varphi > 0$, a patient with a higher recurrent-event frailty also carries a higher death hazard. If $\varphi = 0$, the two processes share no residual risk once treatment is accounted for.

The recurrent-event data are built in counting-process format, one row per at-risk interval. The terminal is one row per patient.

```
set.seed(6)
n_sf <- 500
trt_sf <- rbinom(n_sf, 1, 0.5)

# One shared latent risk per patient, sigma_shared = 0.5
sigma_shared <- 0.5
shared_risk <- rnorm(n_sf, 0, sigma_shared)

# Terminal: death depends on treatment (-0.35) and the shared risk (0.80).
terminal_rate <- 0.12 * exp(-0.35 * trt_sf + 0.80 * shared_risk)
terminal_time <- rexp(n_sf, terminal_rate)
censor_time <- runif(n_sf, 3, 6)
follow_time <- pmin(terminal_time, censor_time)
death_sf <- as.integer(terminal_time <= censor_time)
```

```

# Recurrent: walk each patient's timeline, one row per interval.
recurrent_rows <- vector("list", n_sf)
for(i in seq_len(n_sf)){
  recurrent_rate <- 0.45 * exp(-0.25 * trt_sf[i] + shared_risk[i])
  start <- 0
  rows_i <- list()
  row_id <- 1

  repeat{
    stop <- start + rexp(1, recurrent_rate)
    if(stop < follow_time[i]){
      # interval ending in a recurrence (recurrent = 1)
      rows_i[[row_id]] <- data.frame(
        id=i, tstart=start, tstop=stop,
        recurrent=1, trt=trt_sf[i]
      )
      start <- stop
      row_id <- row_id + 1
    }else{
      # final interval, ending in censoring at end of follow-up (recurrent = 0)
      if(follow_time[i] > start){
        rows_i[[row_id]] <- data.frame(
          id=i, tstart=start, tstop=follow_time[i],
          recurrent=0, trt=trt_sf[i]
        )
      }
      break
    }
  }
  recurrent_rows[[i]] <- do.call(rbind, rows_i)
}

recurrentDat <- do.call(rbind, recurrent_rows) # many rows per patient
terminalDat <- data.frame(                  # one row per patient
  id = seq_len(n_sf),
  time = follow_time,
  death = death_sf,
  trt = trt_sf
)

c(
  recurrent_rows = nrow(recurrentDat),
  recurrent_events = sum(recurrentDat$recurrent),
  deaths = sum(terminalDat$death)
)

```

```

## recurrent_rows recurrent_events deaths
##          1287          787          197

```

The frailty (1 | id) is written only in the recurrent-event formula. The argument `assocSurv = TRUE` shares it into the terminal formula, scaled by the estimated φ . The `truncation = tstart` argument makes each interval a left-truncated risk period.

```

M_shared_frailty <- joint(
  formSurv = list(
    inla.surv(tstop, recurrent, truncation = tstart) ~ trt + (1 | id), # recurrent with frailty
    inla.surv(time, death) ~ trt # terminal, receives the shared frailty
  ),
  dataSurv = list(recurrentDat, terminalDat),
  id = "id",
  basRisk = c("rw1", "rw1"),
  assocSurv = TRUE # share the recurrent frailty into the terminal model
)

summary(M_shared_frailty, sdcor = TRUE)

```

```

##
## Survival outcome (S1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S1 0.0513 0.0405      0.0071  0.0406      0.1595
## trt_S1                -0.1643 0.0844      -0.3299  -0.1642      0.0011
##
## Frailty term standard deviation (S1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## IDIntercept_S1 0.458 0.0478      0.3694  0.4564      0.5572
##
## Survival outcome (S2)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Baseline risk (variance)_S2 0.0773 0.0692      0.0074  0.0575      0.2651
## trt_S2                -0.2950 0.1466      -0.5830  -0.2948      -0.0078
##
## Association survival - survival
##               mean      sd 0.025quant 0.5quant 0.975quant
## IDIntercept_S1_S2 0.6928 0.3455      0.0221  0.6896      1.3826
##
## log marginal-likelihood (integration)      log marginal-likelihood (Gaussian)
##               -4002.472                                -4000.429
##
## Deviance Information Criterion: 7930.282
## Widely applicable Bayesian information criterion: 7928.309
## Computation time: 3.16 seconds

```

- `trt_S1`, the treatment effect on the recurrent stream. Simulated at -0.25.
- `trt_S2`, the treatment effect on death. Simulated at -0.35.
- the frailty standard deviation, how much patients differ in hidden recurrent-event risk. Simulated at 0.50.
- the shared-frailty association, the row linking the two processes, the estimate of φ . Here the association is shown on its original scale, with a true value of $\varphi = 0.80$. A positive value means that a higher recurrent-event frailty also raises the death hazard.

Each of these survival models can in turn serve as the survival component of a joint model, linked to a longitudinal marker through shared random effects.

Survival session summary

- `basRisk` selects a parametric baseline or a flexible random walk baseline.
- The fitted baseline and covariate effects give survival predictions, with prior sensitivity assessed on the

same data.

- Splitting follow-up allows covariates to vary over time.
- Lists of survival formulas handle competing risks. Random effects handle clustering and can link multiple survival models.

The next session introduces longitudinal models, before combining longitudinal and survival processes in joint models.